

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1711

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I P.Divakaran(192424260) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

1. Greedy Best-First Search (GBFS) in this Scenario

Greedy Best-First Search selects the next path based only on the heuristic value $h(n)$, which estimates the remaining distance to the destination. It ignores the actual travel cost already incurred.

In this scenario, the drone uses the straight-line distance to the destination as its heuristic. At every step, it chooses the location that appears closest to the emergency site.

Strengths

- Very fast because it considers only the heuristic.
- Suitable for urgent situations where quick decisions are required.
- Requires less computation and memory than many other search algorithms.

Weaknesses

- Does not guarantee the shortest or safest route.
 - May choose dangerous paths because it ignores travel costs such as bad weather or difficult terrain.
 - If a road suddenly becomes blocked, it may need to search again, causing delays.
 - Can become trapped in poor paths due to inaccurate heuristic estimates.
-

2. A Search in the Same Scenario*

A* Search combines:

- **g(n):** Actual cost from the starting point to the current location.
- **h(n):** Estimated cost from the current location to the goal.

It uses the evaluation function:

f(n) = g(n) + h(n)

The drone considers both the distance already traveled and the estimated remaining distance before choosing the next route.

Advantages

- Produces the optimal route when the heuristic is admissible.
- Considers changing travel costs caused by weather, terrain, or flooded roads.
- Balances speed and safety.
- More reliable than Greedy Best-First Search in uncertain environments.

Disadvantages

- Requires more computation and memory.
- Replanning after environmental changes can increase processing time.

3. Impact of Heuristic Accuracy

The heuristic function greatly influences the performance of both algorithms.

Heuristic Accuracy	Greedy Best-First Search	A* Search
Highly accurate	Finds a good path quickly	Finds the optimal path efficiently
Moderately accurate	May choose suboptimal paths	Usually still finds the optimal solution
Poor or misleading	Performance becomes poor and may get stuck	Slower but still more reliable because actual cost is considered

Therefore, A* is less affected by heuristic errors than Greedy Best-First Search.

4. Effect of Dynamic Changes (Blocked Paths)

Flood conditions constantly change, causing roads to become blocked.

Greedy Best-First Search

- Must frequently restart its search.
- Previously selected paths may become unusable.
- Can waste time exploring blocked routes.

A Search*

- Updates path costs when obstacles appear.
- Recalculates a better alternative route.
- Handles dynamic environments more effectively, although replanning increases computation.

5. Recommended Algorithm

A* is the most suitable algorithm for this scenario.

Justification

- Provides the optimal and safest route.
- Considers both travel cost and estimated remaining distance.
- Adapts better to changing weather and blocked roads.
- Reduces delivery delays while minimizing risk.
- Although it requires more computation, modern drones can perform A* efficiently enough for real-time emergency response.

Greedy Best-First Search is faster but may produce unsafe or inefficient routes because it ignores actual travel costs.

Conclusion

In a flood-affected environment, drones must make fast and safe navigation decisions under uncertainty. Greedy Best-First Search offers rapid decision-making but often sacrifices route quality and safety. A* Search balances actual travel cost with heuristic estimates, making it more reliable and optimal. Therefore, **A*** is the preferred algorithm for emergency medicine delivery because it ensures timely, safe, and efficient navigation even when conditions change dynamically.

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

1. Formulate the Problem as an Optimization Problem

The traffic signal optimization problem aims to find the best signal timings that reduce waiting time, fuel usage, and congestion.

Objective Function

Minimize:

- Total vehicle waiting time
- Fuel consumption
- Traffic congestion

Decision Variables

- Green light duration
- Red light duration
- Signal cycle length
- Traffic signal sequence

Constraints

- Minimum and maximum green signal time
- Pedestrian crossing time
- Emergency vehicle priority
- Coordination between nearby intersections

Why Traditional Search is Inefficient

Traditional search algorithms explore many possible signal combinations. With hundreds of intersections, the number of possible solutions becomes extremely large, making exhaustive search slow and impractical for real-time traffic management.

2. Hill Climbing in this Scenario

Hill Climbing is a local search algorithm that starts with an initial traffic signal plan and repeatedly makes small changes to improve traffic flow.

Working

1. Start with an initial signal timing.
2. Evaluate traffic performance.
3. Modify one or more signal timings.
4. If the new solution is better, accept it.
5. Repeat until no better neighboring solution exists.

Advantages

- Simple and easy to implement.
- Fast for small optimization problems.
- Requires less memory.

Limitations

- May stop at a local optimum instead of finding the best solution.
 - Cannot easily escape poor solutions.
 - Performance depends on the starting point.
-

3. Local Maxima, Plateaus, and Ridges

Local Maxima

A solution that is better than nearby solutions but is not the best overall.

Real-world example:

The signal timings improve traffic in one area while causing congestion elsewhere.

Plateaus

Many neighboring solutions have the same performance.

Real-world example:

Changing the green light by one or two seconds produces no noticeable improvement.

Ridges

The best solution requires multiple changes at the same time, but Hill Climbing changes only one variable at a time.

Real-world example:

Improving traffic flow may require adjusting several nearby intersections together.

4. How Simulated Annealing and Genetic Algorithms Overcome These Problems

Simulated Annealing

Simulated Annealing sometimes accepts worse solutions temporarily to escape local maxima.

Advantages

- Escapes local optima.
- Finds better global solutions.
- Works well in complex search spaces.

Disadvantage

- May require more time than Hill Climbing.
-

Genetic Algorithm (GA)

A Genetic Algorithm is inspired by natural evolution.

Working

1. Generate a population of signal timing solutions.
2. Evaluate each solution using a fitness function.
3. Select the best solutions.
4. Apply crossover and mutation to create new solutions.
5. Repeat until a high-quality solution is found.

Advantages

- Explores many solutions simultaneously.
- Avoids local maxima more effectively.
- Suitable for large and complex optimization problems.
- Easily adapts to changing traffic conditions.

Disadvantages

- More computationally expensive.
- Requires parameter tuning (population size, mutation rate, etc.).

5. Recommended Approach

The **Genetic Algorithm (GA)** is the most suitable approach for smart city traffic optimization.

Justification

- Handles very large search spaces efficiently.
- Optimizes multiple objectives simultaneously.
- Adapts well to changing traffic conditions.
- Produces high-quality solutions without getting trapped in local optima.
- Can optimize traffic across hundreds of intersections at the same time.

Although Hill Climbing is fast, it often gets stuck in local maxima. Simulated Annealing performs better than Hill Climbing, but Genetic Algorithms provide greater scalability and adaptability for large smart city systems.

Conclusion

Traffic signal optimization is a complex optimization problem with many possible solutions. Traditional search methods are inefficient because of the enormous search space. Hill Climbing is simple and fast but suffers from local maxima, plateaus, and ridges. Simulated Annealing improves performance by escaping local optima, while **Genetic Algorithms** offer the best balance of scalability, adaptability, and solution quality. Therefore, **Genetic Algorithms are the most suitable AI technique for smart city traffic signal optimization.**

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

1. Why This Problem Requires an Online Search Agent Instead of Offline Search

An **offline search** requires complete knowledge of the environment before planning a path. Since the Mars rover does not have a complete map of the terrain, offline search is not suitable.

An **online search agent** makes decisions while exploring the environment. It senses the surroundings, updates its knowledge, and chooses the next action based on the latest information.

Reasons for Using Online Search

- The environment is unknown.
- The rover has limited sensing capability.
- New obstacles are discovered during exploration.
- Decisions must be made in real time.
- The rover continuously updates its route as new information becomes available.

2. Challenges of Partial Observability and Dynamic Environment

Partial Observability

The rover can only observe nearby areas using its sensors.

Challenges:

- Unknown obstacles may appear suddenly.
- Hidden craters or rocks increase navigation risk.
- The rover cannot predict the entire path in advance.

Dynamic Environment

Although Mars changes slowly, new information is constantly discovered during exploration.

Challenges:

- Previously planned routes may become unsafe.
- Dust storms or rough terrain can affect movement.
- The rover must frequently update its navigation plan.

3. How the Rover Builds and Updates Its Internal Model

The rover maintains an internal map of the environment.

Working

1. Collect sensor data about nearby terrain.
2. Identify safe and unsafe areas.
3. Update the internal map with newly discovered information.
4. Plan the next movement based on the updated map.
5. Repeat the process until the mission is completed.

Benefits

- Improves navigation accuracy.
- Avoids previously discovered hazards.
- Enables better decision-making as exploration continues.

4. Comparison of Online Search with Uninformed and Informed Search

Feature	Online Search	Uninformed Search	Informed Search
Knowledge of Environment	Partial	Complete	Complete
Uses Heuristic	May use local information	No	Yes
Suitable for Unknown Terrain	Yes	No	No
Adaptability	High	Low	Moderate
Real-Time Decision Making	Yes	No	Limited

Comparison:

- **Uninformed Search** (such as BFS or DFS) assumes the environment is known and does not use heuristic information.
 - **Informed Search** (such as A*) uses heuristics to find efficient paths but also assumes sufficient knowledge of the environment.
 - **Online Search** is the best choice because it explores and plans simultaneously in an unknown environment.
-

5. Recommended Approach

An **Online Search Agent** is the most suitable approach for the Mars rover.

Justification

- Works effectively in unknown environments.
 - Continuously updates its knowledge using sensor data.
 - Adapts to newly discovered obstacles.
 - Supports real-time decision-making.
 - Improves safety by avoiding hazards while exploring.
 - Efficiently balances exploration and navigation.
-

Conclusion

The Mars rover operates in an unknown and partially observable environment where a complete map is unavailable. Therefore, an **Online Search Agent** is more suitable than offline, uninformed, or informed search methods. By continuously sensing the environment, updating its internal model, and adapting its decisions, the rover can safely navigate, avoid hazards, and successfully complete its exploration mission.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

1. Formulate the Problem as a Constraint Satisfaction Problem (CSP)

A **Constraint Satisfaction Problem (CSP)** consists of **variables, domains, and constraints**. The goal is to assign values to variables while satisfying all constraints.

Variables

Each variable represents an exam that needs to be scheduled.

Examples:

- Exam A
- Exam B
- Exam C
- Exam D

Domains

The domain is the set of possible values for each variable.

Examples:

- Time slots (Morning, Afternoon, Evening)
- Exam halls (Hall 1, Hall 2, Hall 3)

Constraints

The timetable must satisfy the following constraints:

- No student should have two exams at the same time.
- The number of students must not exceed the capacity of an exam hall.
- Some subjects must be scheduled before others.

- Faculty members must be available during assigned exam times.
- Students requiring special accommodations must receive suitable exam rooms.

2. Variables, Domains, and Constraints with Explanation

Component	Explanation
Variables	Exams that need to be scheduled.
Domains	Available time slots and exam halls for each exam.
Constraints	Rules that ensure there are no conflicts and all requirements are satisfied.

Example:

- Variable: Mathematics Exam
- Domain: Monday Morning, Monday Afternoon, Tuesday Morning
- Constraint: Cannot be scheduled at the same time as Physics because many students take both subjects.

3. How Backtracking Search Works

Backtracking is a systematic search algorithm used to solve CSPs.

Working

1. Select an exam to schedule.
2. Assign an available time slot and exam hall.
3. Check whether all constraints are satisfied.
4. If the assignment is valid, continue with the next exam.
5. If a conflict occurs, return (backtrack) to the previous assignment and try another option.
6. Repeat until all exams are successfully scheduled.

Advantages

- Guarantees a valid timetable if one exists.
- Simple and effective for solving CSPs.

Disadvantages

- Can become slow for large universities with many courses and students because it may explore many possible combinations.
-

4. How Constraint Propagation (Forward Checking) Improves Efficiency

Forward Checking is a constraint propagation technique that reduces unnecessary searching.

Working

- After assigning a time slot to one exam, the algorithm removes conflicting time slots from the domains of related exams.
- If any exam has no remaining valid time slot, the algorithm backtracks immediately.

Benefits

- Detects conflicts early.
- Reduces the number of possible assignments.
- Speeds up timetable generation.
- Minimizes unnecessary backtracking.

Example:

If the Mathematics exam is scheduled for Monday Morning, then Monday Morning is removed from the available options for all exams taken by the same students.

5. Real-World Challenges and Best Strategy for Scalability

Challenges

- Thousands of students and hundreds of courses.
- Limited number of exam halls.
- Faculty availability conflicts.
- Special accommodation requirements.
- Last-minute changes such as hall unavailability or exam rescheduling.

Best Strategy

The best approach is to use **Backtracking Search with Constraint Propagation (Forward Checking)**.

Justification

- Efficiently handles multiple scheduling constraints.
- Detects conflicts early and reduces unnecessary search.
- Produces valid exam timetables.

- Scales better for large universities than simple backtracking alone.
 - Can adapt to changes by updating only the affected parts of the timetable.
-

Conclusion

Exam timetable generation is a classic **Constraint Satisfaction Problem (CSP)**. The variables are the exams, the domains are the available time slots and exam halls, and the constraints ensure a conflict-free schedule. **Backtracking Search** systematically finds valid assignments, while **Forward Checking** improves efficiency by eliminating invalid choices early. Therefore, **Backtracking combined with Constraint Propagation (Forward Checking)** is the most suitable strategy because it is accurate, scalable, and capable of handling the complex constraints of real-world university scheduling.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

1. How the Minimax Algorithm Models This Problem

The **Minimax algorithm** is used in two-player games where one player tries to maximize the score (AI) while the other tries to minimize it (human opponent).

Working

1. Generate all possible moves from the current game state.
2. Build a game tree representing future moves.
3. Evaluate the possible outcomes.
4. Assume the opponent always makes the best possible move.
5. Choose the move that gives the AI the highest guaranteed score.

Advantages

- Makes intelligent decisions.
 - Finds the optimal move if the entire game tree can be searched.
 - Suitable for competitive strategy games.
-

2. Computational Challenges of Large Game Trees

In real-time strategy games, every move creates many possible future moves, resulting in a huge game tree.

Challenges

- Very high branching factor (many possible moves).
- Large memory requirements.
- Long computation time.
- Difficult to search the entire tree before making a move.
- Real-time games require decisions within seconds.

Therefore, searching the complete game tree is often impractical.

3. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm. It removes branches that cannot influence the final decision.

Working

- **Alpha (α):** Best score the AI can guarantee.
- **Beta (β):** Best score the opponent can guarantee.
- If a branch cannot produce a better result than the current alpha or beta values, it is skipped (pruned).

Benefits

- Reduces the number of nodes evaluated.
- Speeds up decision-making.
- Produces the same optimal result as Minimax.
- Allows deeper searches within the same time limit.
- Improves real-time performance.

4. Design an Evaluation Function

When the search cannot reach the end of the game, the AI uses an **evaluation function** to estimate the quality of a game state.

Factors Considered

- Number and strength of AI units.
- Health of units.
- Available resources (gold, food, energy, etc.).
- Control of important locations.
- Distance from enemy base.
- Defensive strength.
- Current score or mission progress.

Example Evaluation Function

Evaluation Score =

- $+50 \times$ Number of AI units
- $+30 \times$ Resource level
- $+40 \times$ Territory controlled
- $-20 \times$ Enemy advantage
- $-10 \times$ Damage taken

A higher score represents a stronger game position for the AI.

5. Recommended Strategy

The best approach is to use **Minimax with Alpha-Beta Pruning and an Evaluation Function**.

Justification

- Makes intelligent and strategic decisions.
- Greatly reduces unnecessary computations.
- Maintains the same optimal decisions as Minimax.
- Meets real-time decision-making requirements.
- Allows deeper game-tree exploration within limited time.
- Improves the AI's performance against human players.

Conclusion

The **Minimax algorithm** enables the AI to choose the best move by considering both its own actions and the opponent's responses. However, large game trees make full Minimax search computationally expensive. **Alpha-Beta Pruning** improves efficiency by eliminating unnecessary branches while still producing the same optimal decisions. Combined with a well-designed **evaluation function**, this approach provides an effective balance between **optimality, speed, and real-time performance**, making it the most suitable strategy for AI opponents in real-time strategy games.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

